Compte-rendu – MERN Séance 3

Persistance des données avec MongoDB Atlas et Mongoose

Mohamed Jegham – 5ème DS G3

École Polytechnique de Sousse – Dr. Abdelweheb GUEDDES – 2026–2027

---

1. Réponse à l'encadré « Prédisez »

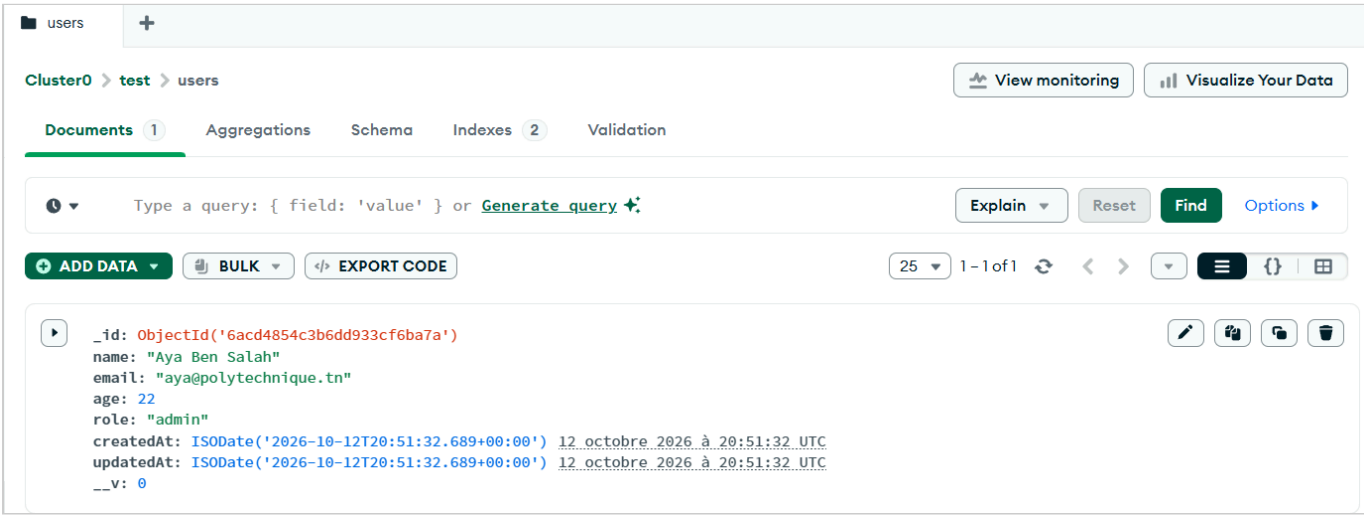
10 requêtes POST, puis redémarrage du serveur (Ctrl+C puis `npm run dev`) : que renvoie `GET /api/articles` ?

En Séance 2, les articles sont stockés dans la variable `let articles = [...]` du contrôleur, donc dans la **mémoire vive du processus Node**. Les 10 POST ajoutent bien 10 objets au tableau (ids 4 à 13), mais Ctrl+C termine le processus et le système libère toute sa mémoire.

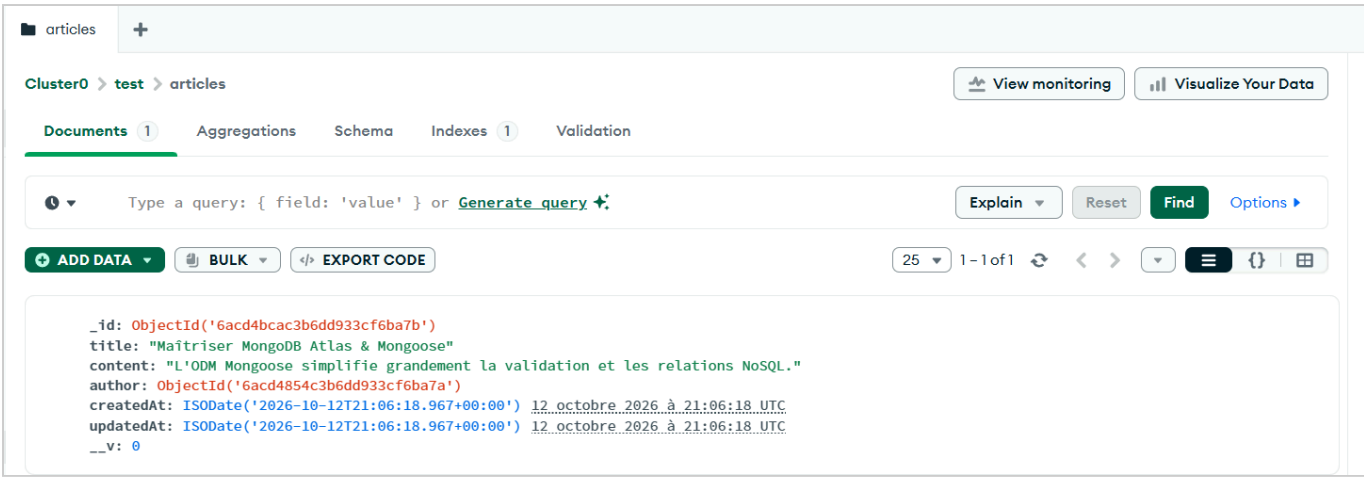
Au redémarrage, Node relit et ré-exécute `articleController.js` : le tableau est recréé à partir de sa valeur initiale écrite dans le code. Le client reçoit donc **200 OK avec uniquement les 3 articles codés en dur** (`total: 3`). Les 10 articles créés sont définitivement perdus, et `prochainId` repart à 4 : le prochain article créé réutilisera un id déjà distribué avant le redémarrage.

Avec MongoDB Atlas, les documents sont écrits sur le disque du cluster, en dehors du processus Node. Redémarrer le serveur ne fait que rouvrir la connexion : `Article.find()` renvoie toujours les 10 articles (vérifié à l'étape 7.4).

2. MongoDB Atlas – collections `users` et `articles`

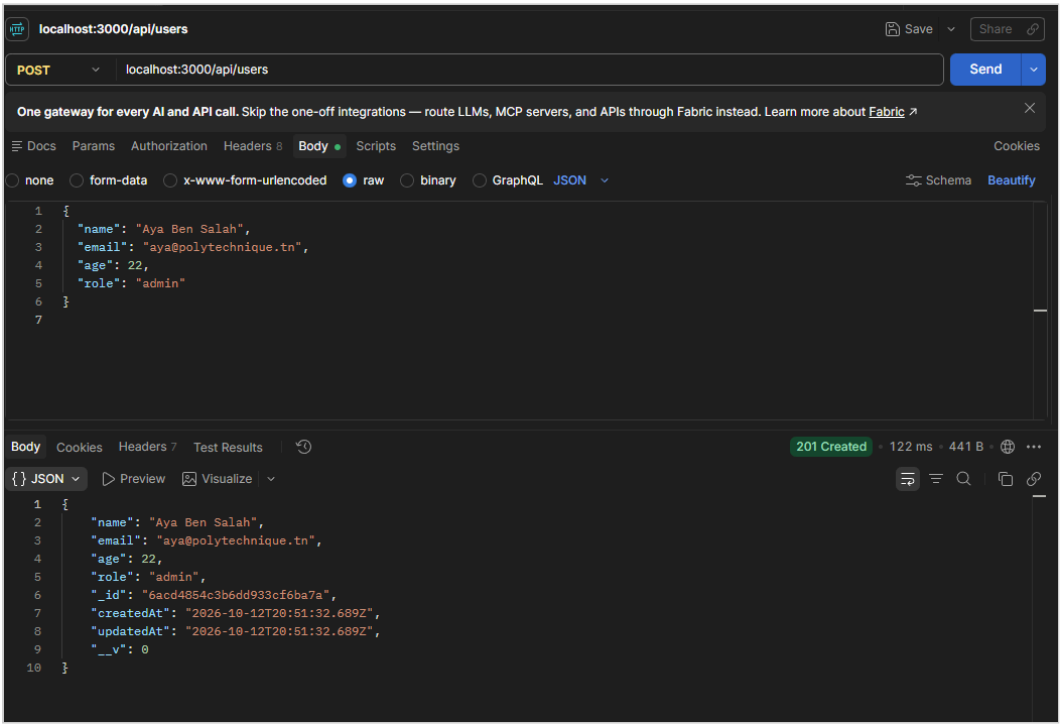


MongoDB Atlas → Database → Collections : base `blogDB`, collection `users` avec les documents insérés (champs `createdAt`/`updatedAt` ajoutés par `timestamps`).

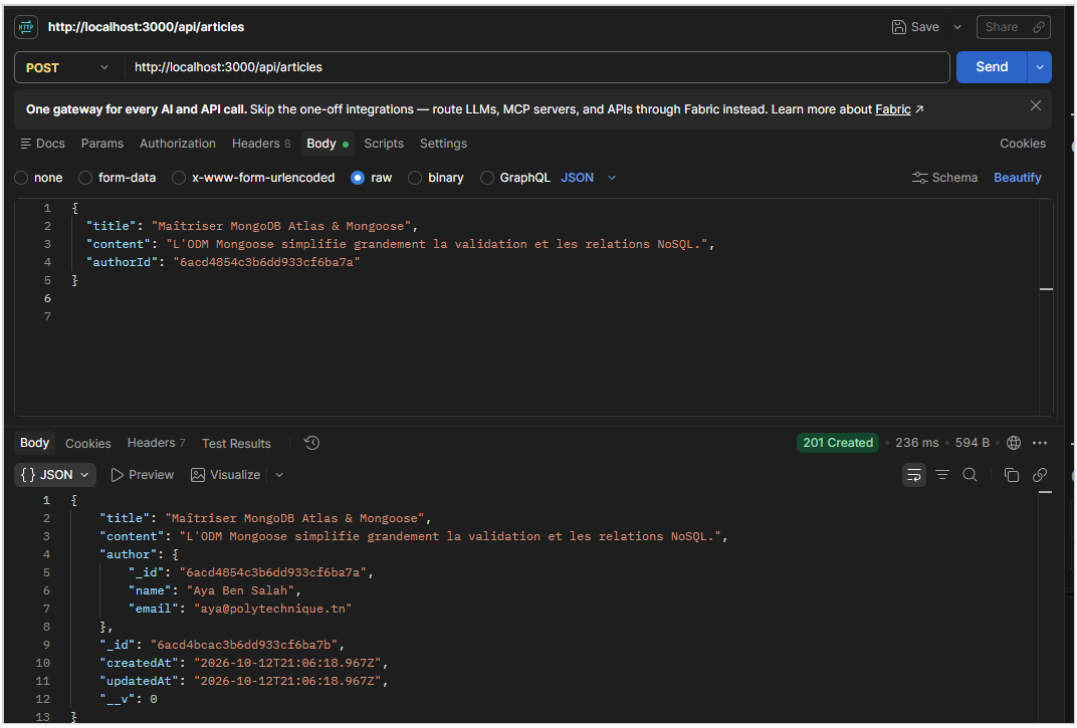


Collection `articles` : le champ `author` est stocké sous forme d'`ObjectId` (référence vers `users`).

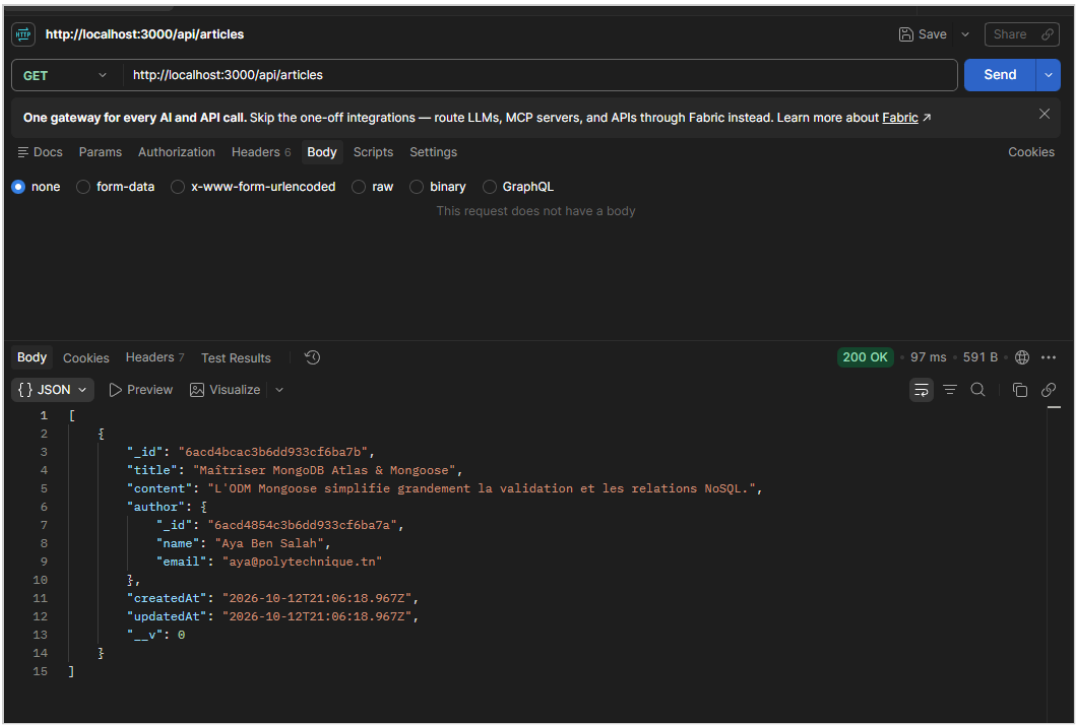
3. Tests Postman – TP guidé



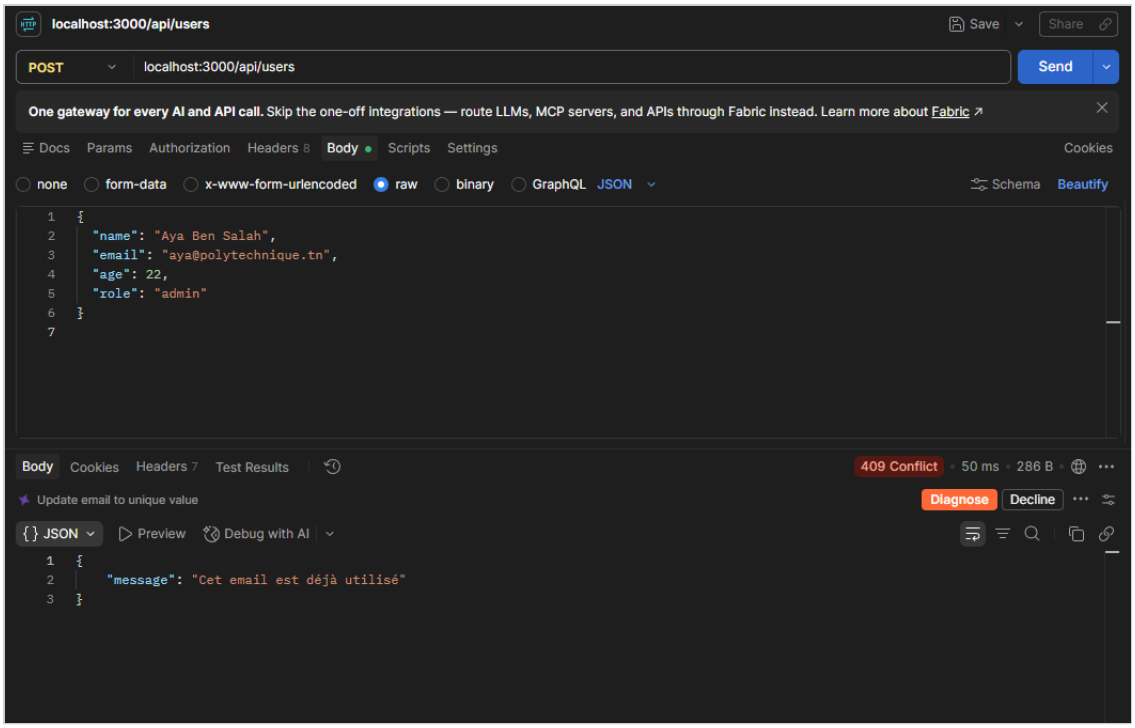
POST /api/users avec un utilisateur valide → **201 Created**. MongoDB génère l'`_id`, le rôle et les dates.



POST /api/articles avec `authorId` = `_id` de l'utilisateur → **201 Created**. Le champ `author` est déjà peuplé (`name`, `email`).



GET /api/articles → **200 OK**. `.populate('author', 'name email')` remplace l'ObjectId par l'objet auteur.



POST /api/users avec un email déjà utilisé → **409 Conflict** (`User.findOne({ email })` trouve un doublon).

4. Exercice 1 – Intégrité référentielle et contraintes avancées

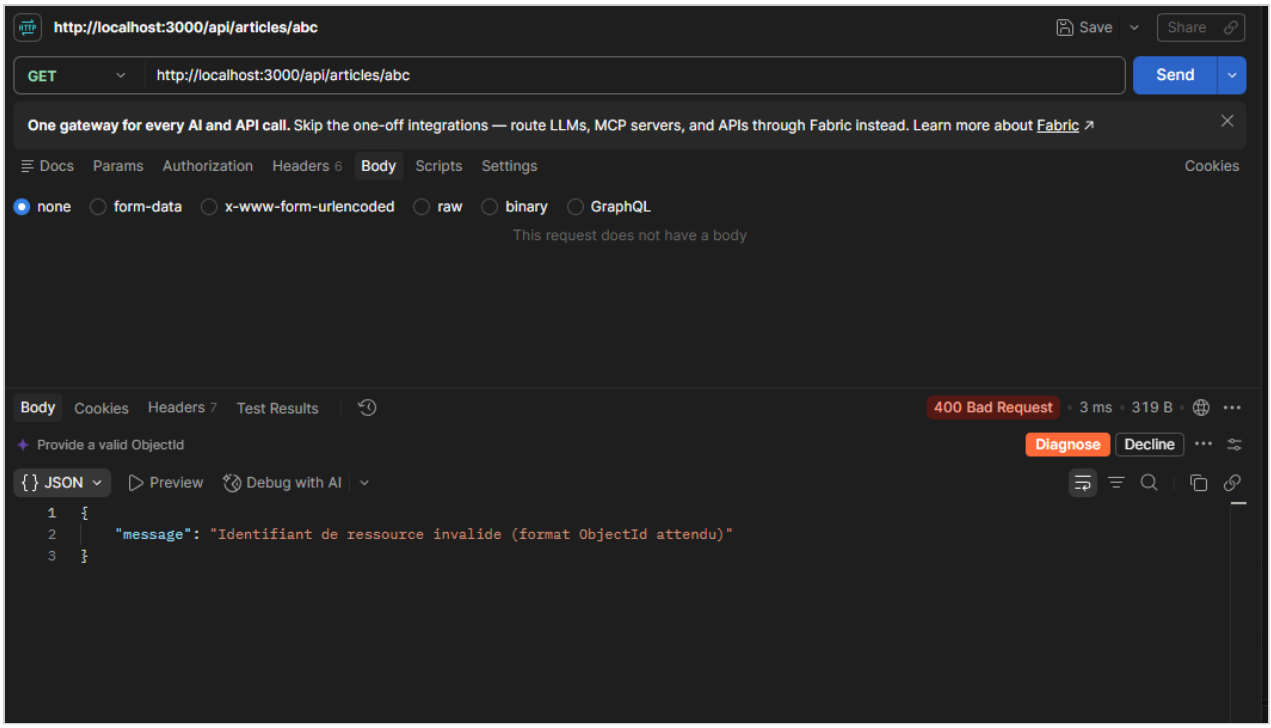
Ajout de `bio` (`maxlength: 200`) et `phone` (`match: /^\d{8}$/`) dans `userModel.js`. Le contrôleur `deleteUser` vérifie d'abord que l'utilisateur existe (404 sinon), puis cherche une dépendance avec `Article.findOne({ author: req.params.id })` : si un article existe, la suppression est refusée (400). Sans cette vérification, les articles de l'utilisateur supprimé pointeraient vers un `_id` inexistant et `.populate()` renverrait `author: null`.

5. Exercice 2 – Gestion des identifiants invalides (CastError)

Un `_id` MongoDB est une chaîne hexadécimale de 24 caractères. Quand on passe `"abc"` à `findById`, Mongoose ne peut pas le convertir en `ObjectId` et lève une `CastError`. Ce n'est pas une panne du serveur mais une erreur du client : la bonne réponse est **400**, pas 500.

Le test `err.name === 'CastError'` est placé dans `utils/gererErreur.js`, appelé dans le `catch` de tous les contrôleurs (dont `getArticleById` et `getUserById`). Le même helper transforme aussi une `ValidationError` en 400 et une erreur d'index unique (code `11000`) en 409.

Requête	Cas	Statut
GET /api/articles/abc	ID mal formé → CastError	400
GET /api/users/123	ID mal formé → CastError	400
GET /api/users/665a1b2c3d4e5f6a7b8c9d0e	ID valide mais inexistant	404



GET /api/articles/abc → **400 Bad Request** « Identifiant de ressource invalide (format ObjectId attendu) » au lieu d'une 500.

6. Code source

models/userModel.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: [true, 'Le nom est obligatoire'],
            trim: true
        },

        email: {
            type: String,
            required: [true, "L'email est obligatoire"],
            unique: true,
            lowercase: true,
            trim: true,
            match: [
                /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
                "Format d'email invalide (ex. : aya@polytechnique.tn)"
            ]
        },

        age: {
            type: Number,
            min: [4, "L'âge minimum autorisé est de 4 ans"]
        },

        role: {
```

```
            type: String,
            enum: {
                values: ['user', 'admin'],
                message: "{VALUE} n'est pas un rôle autorisé"
            },
            default: 'user'
        },

        // Exercice 1 : biographie optionnelle limitée à 200 caractères
        bio: {
            type: String,
            trim: true,
            maxlength: [200, 'La biographie ne peut pas dépasser 200 caractères']
        },

        // Exercice 1 : téléphone validé par regex (8 chiffres)
        phone: {
            type: String,
            trim: true,
            match: [/^\d{8}$/, 'Numéro de 8 chiffres requis']
        }
    },
    { timestamps: true } // Ajoute automatiquement createdAt et updatedAt
);

module.exports = mongoose.model('User', userSchema);
```

models/articleModel.js

```
const mongoose = require('mongoose');

const articleSchema = new mongoose.Schema(
    {
        title: {
            type: String,
            required: [true, 'Le titre est obligatoire'],
            trim: true
        },

        content: {
            type: String,
            required: [true, 'Le contenu est obligatoire']
        },

        author: {
            type: mongoose.Schema.Types.ObjectId,
            ref: 'User',
            required: [true, "L'auteur (authorId) est obligatoire"]
        }
    },
    { timestamps: true }
);

module.exports = mongoose.model('Article', articleSchema);
```

controllers/userController.js

```
const User = require('../models/userModel');
const Article = require('../models/articleModel');
const gererErreur = require('../utils/gererErreur');

// 1. Récupérer tous les utilisateurs (GET /api/users, filtre optionnel ?role=...)
const getAllUsers = async (req, res) => {
    try {
        const { role } = req.query;
        const users = await User.find(role ? { role } : {});
        res.status(200).json(users);
    } catch (err) {
        gererErreur(err, res);
    }
};

// 2. Récupérer un utilisateur par son identifiant (GET /api/users/:id)
const getUserById = async (req, res) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) {
            return res.status(404).json({ message: 'Utilisateur non trouvé' });
        }
        res.status(200).json(user);
    } catch (err) {
        gererErreur(err, res); // CastError -> 400 (Exercice 2)
    }
};

// 3. Créer un nouvel utilisateur (POST /api/users)
const createUser = async (req, res) => {
    try {
        const { name, email, age, role, bio, phone } = req.body;
        if (!name || !email) {
            return res.status(400).json({ message: "Le nom et l'email sont requis" });
        }

        // Vérification de doublon d'email
        const exists = await User.findOne({ email: email.toLowerCase().trim() });
        if (exists) {
            return res.status(409).json({ message: 'Cet email est déjà utilisé' });
        }

        const user = await User.create({ name, email, age, role, bio, phone });
        res.status(201).json(user);
    } catch (err) {
        gererErreur(err, res); // ValidationError -> 400
    }
};

// 4. Supprimer un utilisateur (DELETE /api/users/:id) - Exercice 1
const deleteUser = async (req, res) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) {
```

```
            return res.status(404).json({ message: 'Utilisateur introuvable pour la
suppression' });
        }

        // Intégrité référentielle : interdit si l'utilisateur a au moins un article
        const article = await Article.findOne({ author: req.params.id });
        if (article) {
            return res.status(400).json({
                message: 'Impossible de supprimer un utilisateur ayant des articles publies'
            });
        }

        await User.findByIdAndDelete(req.params.id);
        res.status(200).json({ message: 'Utilisateur supprimé avec succès', id: req.params.id
});
    } catch (err) {
        gererErreur(err, res);
    }
};

module.exports = {
    getAllUsers,
    getUserById,
    createUser,
    deleteUser
};
```

controllers/articleController.js

```
const Article = require('../models/articleModel');
const User = require('../models/userModel');
const gererErreur = require('../utils/gererErreur');

// 1. Récupérer tous les articles avec les données de l'auteur (filtre optionnel ?author=<id>)
const getAllArticles = async (req, res) => {
    try {
        const { author } = req.query;
        const articles = await Article.find(author ? { author } : {}).populate('author', 'name
email');
        res.status(200).json(articles);
    } catch (err) {
        gererErreur(err, res);
    }
};

// 2. Récupérer un article unique par son ID (GET /api/articles/:id)
const getArticleById = async (req, res) => {
    try {
        const article = await Article.findById(req.params.id).populate('author', 'name
email');
        if (!article) {
            return res.status(404).json({ message: 'Article non trouvé' });
        }
        res.status(200).json(article);
    } catch (err) {
        gererErreur(err, res); // CastError -> 400 (Exercice 2)
```

```
    }
};

// 3. Créer un nouvel article lié à un auteur (POST /api/articles)
const createArticle = async (req, res) => {
    try {
        const { title, content, authorId } = req.body;
        if (!title || !content || !authorId) {
            return res.status(400).json({ message: 'title, content et authorId sont
obligatoires' });
        }

        // Vérifier l'existence réelle de l'auteur dans la collection 'users'
        const author = await User.findById(authorId);
        if (!author) {
            return res.status(404).json({ message: 'Auteur non trouvé (identifiant authorId
invalide)' });
        }

        const newArticle = await Article.create({ title, content, author: authorId });
        const populatedArticle = await newArticle.populate('author', 'name email');
        res.status(201).json(populatedArticle);
    } catch (err) {
        gererErreur(err, res);
    }
};

// 4. Mettre à jour un article (PUT /api/articles/:id)
const updateArticle = async (req, res) => {
    try {
        const { title, content } = req.body;
        const article = await Article.findByIdAndUpdate(
            req.params.id,
            { title, content },
            // returnDocument: 'after' remplace { new: true } (déprécié dans Mongoose 9)
            { returnDocument: 'after', runValidators: true }
        );
        if (!article) {
            return res.status(404).json({ message: 'Article introuvable pour la mise à jour'
});
        }
        const populated = await article.populate('author', 'name email');
        res.status(200).json(populated);
    } catch (err) {
        gererErreur(err, res);
    }
};

// 5. Supprimer un article (DELETE /api/articles/:id)
const deleteArticle = async (req, res) => {
    try {
        const article = await Article.findByIdAndDelete(req.params.id);
        if (!article) {
            return res.status(404).json({ message: 'Article introuvable pour la suppression'
});
        }
```

```
        res.status(200).json({ message: 'Article supprimé avec succès', id: req.params.id });
    } catch (err) {
        gererErreur(err, res);
    }
};

module.exports = {
    getAllArticles,
    getArticleById,
    createArticle,
    updateArticle,
    deleteArticle
};
```

utils/gererErreur.js

```
// Traduit les erreurs Mongoose en réponses HTTP propres (utilisé dans les catch des
contrôleurs)
const gererErreur = (err, res) => {
    // Exercice 2 : identifiant mal formé (ex. GET /api/articles/abc)
    if (err.name === 'CastError') {
        return res.status(400).json({
            message: 'Identifiant de ressource invalide (format ObjectId attendu)'
        });
    }

    // Contrainte du schéma non respectée (required, match, min, enum, maxlength...)
    if (err.name === 'ValidationError') {
        const erreurs = Object.values(err.errors).map(e => e.message);
        return res.status(400).json({ message: 'Données invalides', erreurs });
    }

    // Index unique violé (ex. email déjà utilisé)
    if (err.code === 11000) {
        return res.status(409).json({ message: 'Cet email est déjà utilisé' });
    }

    res.status(500).json({ message: err.message });
};

module.exports = gererErreur;
```

routes/userRoutes.js

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// Le préfixe '/api/users' est défini dans server.js : '/' représente donc '/api/users'
router.get('/', userController.getAllUsers);
router.get('/:id', userController.getUserById);
router.post('/', userController.createUser);
router.delete('/:id', userController.deleteUser);

module.exports = router;
```

config/db.js

```
const mongoose = require('mongoose');

async function connectDB(uri) {
  if (!uri) {
    throw new Error("L'URI MongoDB est manquant dans les variables d'environnement (.env)");
  }

  await mongoose.connect(uri, {
    autoIndex: true // Génère automatiquement les index (ex: unique) en développement
  });

  console.log('Connexion à MongoDB Atlas réussie avec succès !');
}

module.exports = connectDB;
```

server.js

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const articleRoutes = require('./routes/articleRoutes');
const userRoutes = require('./routes/userRoutes');

// Chargement des variables d'environnement (.env)
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware pour parser le corps des requêtes en JSON
app.use(express.json());

// Montage des routeurs modulaires
app.use('/api/articles', articleRoutes);
app.use('/api/users', userRoutes);

// Connexion asynchrone à MongoDB Atlas puis démarrage de l'écoute
(async () => {
    try {
        // 1. Établissement de la connexion distante
        await connectDB(process.env.MONGODB_URI);

        // 2. Démarrage de l'écoute du serveur HTTP
        app.listen(PORT, () => {
            console.log(`Serveur MERN en écoute sur http://localhost:${PORT}`);
        });
    } catch (err) {
        console.error('Erreur critique de démarrage du serveur :', err.message);
        process.exit(1);
    }
})();
```